`.sillok`: A Korean-Specific Lossless Text Encoding Format for Linguistically-Informed Compression and Long-Term Archival

Jewoo Suh
Independent Researcher
jewoosuh0111 [at] gmail.com

***Abstract*—UTF-8 encodes every Hangul syllable in a fixed three bytes, ignoring the strong frequency structure of the Korean script, while the general-purpose compressors that do exploit that structure produce output recoverable only with a faithful implementation of their algorithm. We present `.sillok`, a lossless encoding for Korean aimed at long-term archival, designed for both efficiency and recoverability. The format decomposes each Hangul syllable into its onset, vowel, and coda and codes each position with a static, published Huffman table derived from a Korean news corpus, encoding a syllable in 9.46 bits on average against UTF-8's 24. A word-length prefix reconstructs word boundaries without storing spaces, a glue marker preserves spacing within mixed-script tokens, and numbers, embedded Latin text, and document structure receive dedicated representations; a date-stamped, scannable day-boundary marker delimits records. Evaluated on a held-out test set of Korean news, with the frequency tables trained on a disjoint split, `.sillok` reduces size by 56.4% relative to UTF-8. On the independent per-record basis for which it is intended, this matches the strongest general-purpose compressor (Brotli) and exceeds gzip, zstd, and lzma, while being the only one of them reconstructible from the bitstream alone, through Korean phonology and frequency analysis. We provide byte-identical Python and C implementations, the latter decoding at 52 times the speed of the former.**

I. Introduction

The Korean writing system, Hangul, possesses a unique compositional structure among the world's scripts. Each syllable block is assembled from exactly three components: an onset consonant (*choseong*, 초성), a nucleus vowel (*jungseong*, 중성), and an optional coda consonant (*jongseong*, 종성), drawn from finite, well-defined inventories of 19, 21, and 28 elements respectively [9], [6]. This regularity, a deliberate product of King Sejong's 15th-century design [10], creates an opportunity that general-purpose text encodings and compression algorithms fail to exploit.

UTF-8, the dominant encoding for Korean text on the internet, allocates a fixed 3 bytes (24 bits) per Hangul syllable [5], [6]. This uniform cost ignores the vast frequency imbalances among Korean phonemes: the vowel ㅏ accounts for nearly 20% of all vowel occurrences in our news corpus, while ㅐ appears in fewer than 0.02% [1]. General-purpose compression algorithms such as gzip [11], zstd [12], and Brotli [13] can exploit statistical redundancy, but they operate on raw byte sequences with no knowledge of Hangul's compositional structure, and they produce output that is opaque to human analysis, a critical limitation for archival applications where long-term decodability without software is desired.

This paper presents `.sillok`, a lossless text encoding format designed specifically for the Korean language. Named after the *Joseon Wangjo Sillok* (조선왕조실록), the daily records of the Joseon Dynasty that were maintained continuously for 472 years and are now a UNESCO Memory of the World [14], the format is motivated by two goals:

1) **Efficient compression** through linguistically-informed encoding that decomposes Hangul syllables into jamo components and applies position-specific Huffman coding, achieving approximately 9.46 bits per syllable compared to UTF-8's 24 bits, a 60.6% reduction at the syllable level.
2) **Archaeological recoverability** as a core design principle: a future reader encountering the raw bitstream should be able to reconstruct the original text through frequency analysis and knowledge of Korean phonology, without access to a software decoder.

The format employs a four-state finite state machine driven by four prefix-free Huffman tables [16]. Rather than encoding each of the 11,172 possible Hangul syllable blocks as an independent symbol, `.sillok` decomposes syllables into their constituent jamo and encodes each position (onset, vowel, coda) with a dedicated Huffman table trained on corpus-derived frequency data. This decomposition reduces the symbol space from 11,172 to a combined 68 symbols across three tables, producing shorter codes and enabling recovery through phonological analysis.

A key innovation is the *word-length prefix* mechanism for

encoding word boundaries. Korean words (어절, spacing units) exhibit a tight syllable-length distribution, with approximately 88.9% of words comprising 1–4 syllables in our news corpus (Section V). Rather than encoding space characters explicitly or appending per-syllable boundary flags, `.sillok` encodes each word's syllable count once at the start, allowing the decoder to count syllables and reinsert boundaries automatically. This costs about 0.85 bits per syllable for word boundaries, below both an entropy-optimal per-syllable boundary flag (0.92 bits) and an explicit space symbol (1.09 bits); a compact `GLUE` marker then preserves spacing within mixed-script tokens such as 제*1*의 losslessly, as detailed in Section IV.

The format additionally provides compact representations for numbers (via binary-coded decimal), embedded non-Hangul text (via a UTF-8 escape marker terminated by a null byte), and document structure (via day-boundary markers and section separators). The day-boundary marker, 16 consecutive zero bits followed by a 21-bit date, exists outside the Huffman encoding system entirely: the long zero run is conspicuous in a raw bit dump and serves as a human-recoverable landmark for locating records, with the following date field providing confirmation. We discuss the scope and limits of this scannability, including false matches that can occur within numeric and ASCII content, in Section VI.

We evaluate `.sillok` on a held-out test set of 2,740 Korean news articles (1.86 million syllables), drawn from a corpus whose separate training split supplies the frequency tables. The format reduces size by 56.4% relative to UTF-8. On the per-record basis for which it is designed, this is competitive with the strongest general-purpose compressors, matching Brotli and exceeding gzip, zstd, and lzma, while remaining recoverable from the bitstream alone in a way none of them are. We provide encoder and decoder implementations in Python and C, verified to produce byte-identical output, and measure decode throughput of 3.74 ms per 100,000 characters in C, a 52× speedup over the Python reference implementation.

The remainder of this paper is organised as follows. Section II reviews Hangul's phonological structure and prior work on Korean text compression. Section III describes the `.sillok` format specification, including the state machine, Huffman tables, and structural markers. Section IV details the word-length prefix mechanism, ASCII escape encoding, and number encoding, with information-theoretic analysis of each. Section V presents compression performance and computational benchmarks. Section VI discusses the archaeological recoverability properties of the format. Section VII concludes with directions for future work.

II. Background

A. Hangul Phonological Structure

Hangul (한글) was created in 1443 and promulgated in 1446 by King Sejong the Great of the Joseon Dynasty as a deliberately engineered writing system, designed to represent the sounds of Korean with a regular, learnable structure [10]. Unlike organically evolved scripts, Hangul encodes phonological features directly into its visual form [8]: each written syllable block is a spatial composition of exactly two or three *jamo* (자모), the atomic letter units of the script.

Each syllable block is assembled from three positional components. The *choseong* (초성), or onset, is the initial consonant; the *jungseong* (중성), or nucleus, is the vowel; and the *jongseong* (종성), or coda, is an optional final consonant. The inventories of each position are fixed: there are 19 distinct onsets, 21 distinct vowels, and 27 distinct coda consonants, with the null coda (no final consonant) treated as a 28th coda symbol. The full space of legal syllable blocks is therefore $19 \times 21 \times 28 = 11{,}172$ distinct characters, all encoded contiguously in Unicode at code points U+AC00 through U+D7A3 [9], [7].

The Unicode decomposition of a syllable block at code point c is given by [7]:

$$\text{onset index} = \left\lfloor \frac{c - \texttt{0xAC00}}{21 \times 28} \right\rfloor \tag{1}$$

$$\text{vowel index} = \left\lfloor \frac{(c - \texttt{0xAC00}) \bmod (21 \times 28)}{28} \right\rfloor \tag{2}$$

$$\text{coda index} = (c - \texttt{0xAC00}) \bmod 28 \tag{3}$$

This decomposition is the algebraic foundation of the `.sillok` encoding: rather than treating each of the 11,172 syllable blocks as an independent symbol, the encoder decomposes each syllable and codes the three components separately.

The motivation for this decomposition is the highly non-uniform frequency distribution of jamo within each positional class. We derive position-specific frequencies from the training split of a 15.1-million-syllable Korean news corpus (Section V); the resulting distribution is broadly consistent with prior reports such as KREF [1]. In our corpus the vowel ㅏ alone accounts for 19.96% of all vowel occurrences, while the four rarest vowels (ㅒ, ㅖ, ㅙ, and ㅟ) together account for about 0.8%. Similarly, the null coda accounts for 51.88% of all coda positions, reflecting the large proportion of open syllables in Korean. This skew makes each positional class highly amenable to Huffman coding: a position-specific Huffman code assigns short codewords to the frequent symbols and longer codewords to the rare ones, whereas a flat encoding of the full 11,172-symbol space wastes bits by treating all syllables as equally probable.

Beyond individual syllables, Korean text is segmented into *eojeol* (어절), spacing units that roughly correspond to morphological words. Eojeol exhibit a tight syllable-length distribution: in our news corpus approximately 88.9% of eojeol contain between one and four syllables, with the three- and two-syllable eojeol being the most common and an average length of 2.96 syllables. This distribution motivates the word-length prefix mechanism described in

Section IV, which encodes the syllable count of each eojeol once at the start rather than marking word boundaries after every syllable.

B. Prior Work on Korean Text Compression

a) Character encodings.: Before the dominance of Unicode, Korean text was most commonly stored in legacy encodings such as EUC-KR and CP949, variable-width encodings that use a single byte for ASCII and two bytes for each precomposed Hangul syllable [4]. UTF-8, now the de facto standard for Korean text on the web, trades this compactness for universality: every Hangul syllable in the U+AC00 to U+D7A3 range requires three bytes [5], [6]. Neither encoding exploits the sub-character structure of Hangul. Both treat the syllable block as an atomic unit and allocate a fixed width to it regardless of how frequently its constituent jamo occur. The `.sillok` format departs from this tradition by encoding below the syllable level, at the granularity of individual jamo.

b) General-purpose compression.: Algorithms such as gzip (DEFLATE) [11], zstd [12], and Brotli [13] achieve substantial size reductions on Korean text by exploiting statistical and dictionary-based redundancy at the byte level. These methods are highly effective and, for large documents, will typically outperform a static per-symbol code. However, they share two limitations relevant to our goals. First, they operate on raw byte sequences and have no model of Hangul's compositional structure, so they cannot exploit the position-specific jamo frequency skew directly; they must rediscover it indirectly through byte-level statistics. Second, and more important for archival use, their output is opaque: a compressed stream is not human-interpretable and is recoverable only with a faithful implementation of the decompression algorithm. This conflicts with the archaeological-recoverability goal that motivates `.sillok` (Section VI).

c) Korean-specific compression.: The most directly comparable prior work is PHDCM, the Parallel Hangul Dynamic Coding Method [3], which models Korean text with a three-state transition graph reflecting the onset, vowel, and coda structure of Hangul syllables and reports roughly 3.5 bits per character, with its principal contribution being an efficient parallel implementation evaluated on a 64-processor SIMD machine. PHDCM establishes the central insight that `.sillok` also builds on: that modeling the positional structure of Hangul yields better compression than treating syllables as atomic symbols. The two methods make different tradeoffs. PHDCM uses dynamic, adaptive coding that achieves a strong compression ratio but produces output whose codebook is determined at runtime; recovering the text requires replaying the exact adaptation procedure from the start of the stream, since no fixed codebook is stored. The `.sillok` format instead uses static Huffman tables derived from corpus frequency data, accepting a larger encoded size in exchange for a fixed, publishable codebook that can be reconstructed through frequency analysis. This choice directly serves the recoverability goal of Section VI. The `.sillok` format also extends the model beyond pure Hangul with dedicated handling for word boundaries, numbers, ASCII runs, and document structure (Section IV).

d) Mixed-script text.: Contemporary Korean text routinely interleaves Hangul with Latin-script loanwords, brand names, and technical terms; corpus studies of lexical borrowing document the substantial and growing presence of such material in Korean writing [15]. A practical Korean encoding must therefore handle embedded ASCII gracefully rather than treating it as an error case. The `.sillok` format addresses this with an explicit ASCII-escape mechanism (Section IV), allowing Latin text to be embedded inline without disrupting the jamo-level encoding of surrounding Hangul.

III. Format Specification

This section specifies the `.sillok` format: the byte-level file container, the four Huffman tables that drive the encoding, the decoding state machine, and the structural markers used to delimit documents. All values reported here are taken from the reference implementation, and have been verified to produce byte-identical bitstreams across the Python and C encoders and decoders on Hangul, numeric, and mixed-script text.

A. File Container

A `.sillok` file consists of a fixed 7-byte header followed by the encoded payload. The header has three fields:

- **Magic number** (2 bytes): the ASCII bytes `SL` (`0x53 0x4C`), identifying the file type.
- **Version** (1 byte): the format version, currently `0x01`.
- **Bit count** (4 bytes): the length of the encoded bitstream in bits, stored as a big-endian unsigned 32-bit integer.

The payload follows immediately and contains the Huffman-coded bitstream, padded with zero bits to the next byte boundary. The explicit bit count is necessary because the payload is not self-terminating: the final byte may contain padding bits that must be discarded, and the decoder uses the stored count to know exactly where the bitstream ends.

B. The Four Huffman Tables

Encoding is driven by four static, prefix-free Huffman tables, each built from corpus-derived frequency data. The tables are fixed at specification time and published as part of the format, so any conforming decoder reconstructs identical codes. The reference Python implementation rebuilds the codes from the frequency data at load time, while the C implementation hard-codes the same codes; we have verified that the two agree bit for bit.

Table 0 (dispatcher). The top-level table contains 15 symbols that determine how the following bits are interpreted. It encodes word-length prefixes (`1-syl` through `6-`

`syl` and an extended `7+-syl` escape), the two most common punctuation marks (comma and period), the middle dot (U+00B7), a `GLUE` marker that suppresses an automatically inserted space (Section IV), and control symbols for numbers, ASCII runs, newlines, and section separators. Table I lists the dispatcher codes produced by the reference implementation.

TABLE I
TABLE 0 (DISPATCHER) HUFFMAN CODES.

Symbol	Code	Freq (%)
`3-syl`	`01`	22.32
`2-syl`	`00`	21.48
`4-syl`	`101`	13.52
`GLUE`	`1111`	10.04
`1-syl`	`1110`	8.33
`NUMBER`	`1100`	5.79
`ASCII_START`	`1001`	5.14
`5-syl`	`1000`	5.12
`.`	`11011`	4.64
`6-syl`	`110100`	1.73
`7+-syl`	`1101011`	1.30
· (U+00B7)	`11010101`	0.58
`NEWLINE`	`110101000`	0.01
`,`	`1101010011`	0.01
`SECTION_SEP`	`1101010010`	0.01

`GLUE` is a first-class dispatcher symbol, accounting for about 10% of dispatch events on the news corpus; it suppresses the space the decoder would otherwise insert before a token continuing an eojeol (Section IV). Frequencies are taken from the training split, with symbols absent or near-absent there (the standalone comma, the newline, and the section separator, all rare in single-paragraph articles) floored to a small nonzero value so that any Korean text remains encodable.

Tables 1–3 (jamo). The remaining three tables encode the components of a Hangul syllable: Table 1 covers the 19 onsets, Table 2 the 21 vowels, and Table 3 the 28 coda values (27 consonants plus the null coda). Because the null coda alone accounts for 51.88% of coda positions, Table 3 assigns it the single-bit code `1`; the most frequent vowel ㅏ (19.96%) receives the 2-bit code `00`. The complete jamo tables are given in Appendix A.

Over the corpus frequencies, the combined jamo tables encode a Hangul syllable in an average of 9.46 bits, against a measured per-syllable entropy of 9.35 bits, for a coding efficiency of 98.8%. Relative to UTF-8's fixed 24 bits per syllable, this represents a 60.6% reduction at the syllable level. End-to-end compression on running news text is lower, because numbers, embedded ASCII, and structural markers add overhead that is not amortised over syllables; we quantify this in Section V.

C. Decoding State Machine

Decoding proceeds as a finite state machine with four states, driven by the dispatcher table. The decoder begins in the WORD-START state.

1) **Word-Start.** Read one symbol from Table 0. A word-length symbol n initialises a syllable counter to n and transitions to ONSET. Control symbols are handled directly: `NEWLINE` emits a line break, punctuation symbols emit their character, `GLUE` suppresses the space before the next token, `NUMBER` and `ASCII_START` enter their respective sub-modes, and `SECTION_SEP` begins a section header.
2) **Onset.** Read one symbol from Table 1 (onset), then transition to VOWEL.
3) **Vowel.** Read one symbol from Table 2 (vowel), then transition to CODA.
4) **Coda.** Read one symbol from Table 3 (coda) and compose the completed syllable. Decrement the syllable counter. If the counter is nonzero, return to ONSET for the next syllable of the same word; otherwise return to WORD-START.

Spaces are not encoded directly. Instead the decoder inserts a single space before each content token (word, number, or ASCII run) by default, and a `GLUE` marker suppresses that space wherever the source had none, as between the pieces of the mixed token 제*1*의. Because consecutive Hangul words form separate tokens, the common case of space-separated eojeol incurs no `GLUE` markers at all. Newlines are preserved exactly. The format is therefore lossless up to a single normalisation: a run of intra-line whitespace is reproduced as one space. This mechanism is analysed in Section IV.

D. Structural Markers

Two markers provide document structure above the level of individual words.

Day boundary. A day boundary is encoded as 16 consecutive zero bits followed by a 21-bit date field: 12 bits of year, 4 bits of month, and 5 bits of day, for 37 bits total. The marker is deliberately placed outside the Huffman system. During decoding it is tested only at dispatch points, that is, in the WORD-START state before a Table 0 symbol is read: the decoder inspects the next 16 bits, and if they are all zero, consumes the 37-bit marker and emits the date; otherwise it proceeds to read a Table 0 code. To prevent a 16-zero run occurring inside ordinary content from being mistaken for a boundary, the decoder validates the trailing date field, accepting the marker only when the month lies in 1–12 and the day in 1–31; the encoder, in turn, emits boundaries only at the start of a record. Across the full held-out test set (1.86 million syllables) this produced no false detections. The long zero run additionally serves as a visual landmark in a raw bit dump, although a naive scan of the *entire* bitstream, rather than the decoder's dispatch points, can still encounter comparable zero runs inside numeric and ASCII payloads; we return to the recoverability implications in Section VI.

Section separator. Within a day's record, the `SECTION_SEP` symbol from Table 0 introduces a thematic section. It is followed by the section name, encoded as

ordinary Hangul words, and terminated by a `NEWLINE`. The reference daily-record format uses eight fixed sections (정치, 경제, 사회, 국제정세, 과학기술, 문화, 천재지변, 사신왈), mirroring the thematic organisation of the historical Joseon annals.

IV. Optimization Techniques

Beyond per-syllable jamo coding, `.sillok` applies three techniques that handle the non-Hangul aspects of real text: word boundaries, numbers, and embedded non-Hangul content. This section describes each and analyses its cost. All per-syllable figures are measured on the training corpus of Section V (15.1 million syllables across 5.1 million words, averaging 2.96 syllables per word).

A. Word-Length Prefix

Korean is written with spaces between *eojeol*, so any encoding must record where each word ends. Three strategies are available:

- **Explicit space:** emit a dedicated space symbol between words.
- **Per-syllable boundary flag:** after each syllable, emit a bit indicating whether the word continues.
- **Word-length prefix (`.sillok`):** emit the word's syllable count once, at its start; the decoder counts that many syllables and then inserts a boundary.

We compare the three at the level of each mechanism's own optimal code. With an average word length of 2.96 syllables, a word boundary follows a fraction $p \approx 0.338$ of syllables. A per-syllable flag therefore encodes a Bernoulli(p) event per syllable, whose entropy lower bound is

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p) \approx 0.92$$

bits per syllable, a bound that a naive one-bit flag (1.000 bits per syllable) does not reach. Encoding spaces as an explicit symbol in the boundary alphabet costs 1.09 bits/syllable, because the added symbol also lengthens the surrounding codes. The word-length prefix costs 0.85 bits/syllable (Table II).

TABLE II
Per-syllable cost of word-boundary encoding strategies.

Strategy	Bits/syllable
Explicit space symbol	1.09
Naive one-bit flag	1.00
Entropy-optimal boundary flag	0.92
Word-length prefix (`.sillok`)	0.85

The word-length prefix thus matches the efficiency of an optimally-coded per-syllable boundary indicator and slightly betters it, because it encodes the full word-length distribution (Table 0 assigns the shortest codes to the most common two- and three-syllable words) rather than treating each syllable as an independent event, and it spends one symbol per word instead of one per syllable. These are mechanism-intrinsic costs under an optimal code; in the deployed format the word-length codes share Table 0 with the dispatch and control symbols, so they realise a somewhat higher per-syllable cost.

B. Lossless Spacing and the GLUE Marker

The word-length prefix reconstructs the boundary between consecutive Hangul words for free: in pure Hangul text every eojeol is a single word token, and the decoder inserts one space before each. Mixed tokens break this assumption. The eojeol 제*1*의, for instance, decomposes into a word (제), a number (1), and a word (의) with no spaces between them, yet the decoder would by default insert a space before each content piece.

To remain lossless, `.sillok` emits a `GLUE` marker before any content piece that continues an eojeol, suppressing the default space. Because gluing occurs only inside mixed tokens, pure Hangul text incurs no `GLUE` markers. On our news corpus `GLUE` accounts for about 10% of dispatch events (news is rich in numbers and Latin-script terms) and adds roughly 0.18 bits/syllable. Combining the prefix and `GLUE`, the word-boundary mechanism costs about 1.03 bits/syllable while reproducing all inter-token spacing exactly, still below the 1.09 bits/syllable of an explicit space symbol. Spacing is preserved up to a single normalisation: a run of intra-line whitespace is reproduced as one space, and newlines are preserved exactly.

C. Number Encoding

A number is encoded as the `NUMBER` marker (5 bits), an 8-bit count of the following characters, and one 4-bit nibble per character. Digits 0–9 use their binary value; the decimal separators `.` and `,` use the otherwise-unused nibble values 10 and 11, so they survive the round trip. A c-character number therefore occupies $13 + 4c$ bits, against $8c$ bits in UTF-8. The format is more compact for numbers of four or more characters, which covers years, prices, and statistics; one- to three-character numbers carry a small fixed overhead. The 8-bit count admits up to 255 characters per number token, removing the silent overflow that a 4-bit count would suffer on long digit strings. Binary-coded decimal spends 4 bits per symbol against an entropy of $\log_2 12 \approx 3.58$ bits for the twelve-symbol alphabet, trading about 0.4 bits per symbol for a fixed, trivially recoverable mapping.

D. Non-Hangul Escape

Characters that are neither Hangul, digits, nor the three dedicated punctuation marks are encoded through an escape: the `ASCII_START` marker (6 bits), the characters' raw UTF-8 bytes (8 bits each), and a terminating null byte. Because UTF-8 never produces a zero byte for a non-null character, the null terminator is unambiguous. A run of k bytes costs $14 + 8k$ bits, exactly 14 bits more than the $8k$ bits the same bytes occupy in UTF-8. This mechanism is therefore not a compression technique but a means of embedding arbitrary non-Hangul content,

such as the Latin-script loanwords and symbols common in contemporary Korean text (Section II), without loss. Grouping consecutive non-Hangul characters into a single run amortises the fixed overhead, and the embedded bytes remain directly readable in a raw dump.

V. Evaluation

A. Corpus and Methodology

We evaluate on a corpus of contemporary Korean news articles [2], using full article bodies rather than headlines. We adopt the corpus's own split: a training set of 22,194 articles (15.1 million Hangul syllables) and a held-out test set of 2,740 articles (1.86 million syllables). All four Huffman tables (Section III) are derived *solely* from the training split, and every figure below is measured on the held-out test set. The evaluation therefore reflects generalisation to unseen text rather than fit to the data used to build the tables. News is a deliberately demanding domain: it interleaves Hangul with frequent numbers, dates, percentages, and Latin-script terms, none of which the jamo coding can compress.

B. Compression Performance

Table III reports total encoded size on the held-out set. `.sillok` reduces the UTF-8 size by 56.4%. This is below the 60.6% achieved by the jamo tables on pure syllables (Section III): the gap is the cost of non-Hangul content. Counting all numbers, ASCII, punctuation, and structural markers, the format spends 12.08 bits per Hangul syllable on running news, against the 9.46-bit jamo cost, because numbers and Latin terms consume bits while adding no syllables to the denominator.

TABLE III
Size reduction relative to UTF-8 on the held-out test set, comparing `.sillok` against four general-purpose compressors in two regimes: each article compressed independently (per-record) and the whole corpus compressed as one stream.

	Reduction vs UTF-8	
Codec	Per-record	Whole corpus
`.sillok`	**56.4%**	n/a
gzip [11]	52.6%	64.4%
zstd [12]	52.9%	77.3%
Brotli [13]	56.6%	77.2%
lzma	49.1%	77.6%

Table III compares `.sillok` against four general-purpose compressors in two regimes. On bulk text the modern compressors are far ahead, reaching roughly 77%, because they exploit redundancy across documents that a static per-symbol code cannot. `.sillok` does not compete in this regime, and is not intended to: it targets independent daily records, not bulk archives.

The per-record regime is the one that matches the format's purpose, and there the picture is different. `.sillok` reduces each record by 56.4%, ahead of gzip (52.6%), zstd (52.9%), and lzma (49.1%), and within a fraction of a point of Brotli (56.6%), the strongest of the four. Brotli reaches this with a large built-in dictionary; `.sillok` matches it with a static model of fewer than a hundred symbols, derived from Korean phonology. The point is not a smaller file than every alternative, but compression comparable to the best of them from a transparent, fixed codebook that, unlike any of these compressors, is recoverable from the bitstream alone (Section VI).

C. Computational Performance

We benchmark both implementations on a 100,000-character news excerpt (Table IV). The C decoder processes 100,000 characters in 3.74 ms and the encoder in 5.62 ms. The Python reference is 52× slower on decode and 15× slower on encode, reflecting its per-bit-string overhead. The two implementations produce byte-identical output, verified on held-out news text.

TABLE IV
Throughput on a 100,000-character news excerpt.

	C	Python	Speedup
Encode (ms)	5.62	84.0	15×
Decode (ms)	3.74	195.9	52×

VI. Archaeological Recoverability

The second design goal of `.sillok` (Section III) is that the format remain readable by a future reader who possesses the raw bitstream but no software decoder and no specification. This property motivates the format's name: the *Joseon Wangjo Sillok* survived for centuries and is still read today [14], whereas a file produced by an adaptive or dictionary-based compressor is intelligible only for as long as a faithful implementation of its algorithm survives. We do not claim that `.sillok` is self-describing. We argue, rather, that a reader equipped with knowledge of Korean phonology and elementary frequency analysis could reconstruct its contents, and we are explicit about where that argument breaks down.

A. Properties that support reconstruction

A static, published codebook. Unlike adaptive schemes such as PHDCM [3], whose codebook changes as the stream is processed and exists only implicitly, `.sillok` uses fixed Huffman tables (Appendix A). The same codeword always denotes the same symbol, so the stream can be treated as a substitution over a small alphabet and attacked with frequency analysis.

Linguistic structure. Hangul's phonotactics are strongly constrained (Section II): syllables are onset-vowel-coda triples, roughly half of all codas are null, and the marginal frequency of each position is stable across corpora. A reader who hypothesises that the stream encodes jamo in fixed positional order can match observed code frequencies against these known distributions: the shortest

codes must be the high-frequency jamo (ㅇ, ㅏ, the null coda) and the long rare codes the rare jamo. The 11,172 syllables collapse onto 68 jamo, an alphabet small enough to solve by hand.

Scannable temporal landmarks. The day-boundary marker (Section III) is sixteen consecutive zero bits followed by a 21-bit date. The long zero run is conspicuous in any rendering of the raw bits, and the date fields increase monotonically through an archive. A reader who notices a recurring zero-run followed by a 21-bit field whose value increments from record to record has strong evidence that the field is a date or counter, without knowing the layout in advance. This monotonicity is the single most useful recovery cue the format provides.

B. A reconstruction sketch

A plausible reconstruction proceeds in stages. First, the reader scans for the conspicuous sixteen-zero runs and reads the 21-bit fields that follow; those that increase monotonically delimit the records and reveal the dating scheme, fixing the year, month, and day sub-fields by their ranges. Second, within a record, frequency analysis on the remaining prefix-free codes recovers the onset, vowel, and coda tables by alignment with known Korean jamo frequencies, and the fixed onset-vowel-coda cycle segments the stream into syllables, which the word-length prefixes group into words. Finally, numeric and embedded-Latin runs, stored as raw binary-coded decimal and UTF-8 bytes rather than Huffman codes, are directly legible once located. The raw representation that makes these runs incompressible (Section IV) also makes them the easiest part of the stream to recover.

C. Limitations

Recoverability is a matter of degree, not a guarantee, and three caveats bound it. First, the day-boundary scan is not collision-free: a sixteen-zero run can occur inside numeric or ASCII payloads (Section III), and although the decoder rejects such matches by validating the date fields, validation alone is weak, since a randomly placed 21-bit field has roughly a three-in-four chance of presenting a plausible month and day. The reliable disambiguator is again monotonicity: spurious matches do not form an increasing sequence at regular intervals, whereas genuine boundaries do. Second, reconstruction presupposes that the reader knows the text is Korean and understands Hangul's compositional structure; the format encodes no metadata announcing this. Third, the procedure we sketch is a human one. The format is recoverable in the sense that the Rosetta Stone was decipherable, through knowledge, frequency analysis, and structural regularity, not in the sense that a generic tool can decode it unaided. We regard this as an honest reading of the goal rather than a weakening of it: the durability of meaning, as the Joseon annals demonstrate, has historically rested on exactly these properties.

VII. Conclusion

We have presented `.sillok`, a lossless text encoding format that exploits the compositional regularity of Hangul. By decomposing each syllable into onset, vowel, and coda and coding each position with a static, corpus-derived Huffman table, the format encodes Korean syllables in 9.46 bits on average, a 60.6% reduction relative to UTF-8 at the syllable level. A word-length prefix reconstructs word boundaries without storing spaces, a `GLUE` marker preserves spacing inside mixed-script tokens, and dedicated representations handle numbers, embedded non-Hangul text, and document structure. We provide byte-identical Python and C implementations and a day-boundary marker designed to serve as a scannable temporal landmark for long-term archival.

Evaluated on a held-out test set of Korean news, with the frequency tables trained on a disjoint split, `.sillok` reduces size by 56.4% relative to UTF-8. It is not a general-purpose compressor: compressed as a bulk corpus, modern methods such as zstd and Brotli are far ahead. On the independent daily records for which the format is intended, however, `.sillok` is competitive with the best of them, matching Brotli and beating gzip, zstd, and lzma, while being the only one of the five reconstructible from the bitstream alone. Its contribution is therefore not a new high-water mark in ratio but a format that pairs competitive per-record compression with linguistic transparency and archaeological recoverability.

Several directions remain. The day-boundary marker relies on date validation and the monotonicity of dates to resolve the zero-run collisions noted in Section VI; a marker provably free of such collisions, for example a reserved synchronisation pattern or a bit-stuffing rule, would make automated recovery as reliable as the human recovery we describe. The static tables are tuned to the news domain, and other registers, such as literary or conversational Korean, would merit their own measured frequencies or a light, recoverability-preserving adaptivity. Finally, the static order-0 jamo model leaves redundancy unexploited: prior adaptive methods reach roughly 3.5 bits per character [3], and a context model that preserves the format's static, recoverable codebook is an open and appealing problem. To keep the format fully specified by this paper, the complete frequency tables (Appendix A) and a worked encoding example (Appendix A) are included, so that an independent reader can both reimplement the codec and exercise the recoverability argument of Section VI.

References

[1] KREF Korean Frequency Analysis. http://www.kref.altervista.org/kreffrequency.html. Corpus of 17,691 Korean syllables with jamo-level frequency data.

[2] D. Kim. *Naver News Summarization* (*Korean*). Hugging Face dataset, 2022. Apache-2.0 licence; full Korean news article bodies, 22,194 training and 2,740 test documents. https://huggingface.co/datasets/daekeun-ml/naver-news-summarization-ko.

[3] Min, Yong-Sik (1995). PHDCM: Efficient Compression of Hangul Text in Parallel. *The Journal of the Acoustical Society of Korea*, 14(2E), 50–56. Parallel Hangul Dynamic Coding Method using a three-state transition graph; reports approximately 3.5 bits per character.

[4] U. Choi, K. Chon, and H. Park (1993). *RFC 1557: Korean Character Encoding for Internet Messages*. Internet Engineering Task Force. EUC-KR encoding using KS C 5601 (KS X 1001); two bytes per Hangul syllable. https://www.rfc-editor.org/rfc/rfc1557.

[5] F. Yergeau (2003). *RFC 3629: UTF-8, a transformation format of ISO 10646*. Internet Engineering Task Force. Characters in U+0800–U+FFFF, including all Hangul syllables, encode as three bytes. https://www.rfc-editor.org/rfc/rfc3629.

[6] The Unicode Consortium. *Hangul Syllables*, Unicode code chart, range U+AC00–U+D7A3. https://www.unicode.org/charts/PDF/UAC00.pdf.

[7] The Unicode Consortium. *The Unicode Standard, Version 16.0.0*, Section 3.12: Conjoining Jamo Behavior. South San Francisco: The Unicode Consortium, 2024. ISBN 978-1-936213-34-4. https://www.unicode.org/versions/Unicode16.0.0/core-spec/chapter-3/.

[8] Sampson, G. (1985). *Writing Systems: A Linguistic Introduction*. Stanford University Press. Introduces the term "featural" to characterise Hangul.

[9] National Institute of Korean Language (국립국어원). Official regulator of the Korean language; reference on Hangul orthography and syllable structure. https://www.korean.go.kr/front_eng/.

[10] UNESCO Memory of the World Register. *Hunminjeongum Manuscript*. Inscribed 1997. King Sejong completed the Korean alphabet in 1443 and promulgated it in 1446. https://www.unesco.org/en/memory-world/hunminjeongum-manuscript.

[11] P. Deutsch (1996). *RFC 1952: GZIP File Format Specification version 4.3*. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc1952.

[12] Y. Collet and M. Kucherawy, Eds. (2021). *RFC 8878: Zstandard Compression and the application/zstd Media Type*. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8878.

[13] J. Alakuijala and Z. Szabadka (2016). *RFC 7932: Brotli Compressed Data Format*. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7932.

[14] National Institute of Korean History. *Joseon Wangjo Sillok* (조선왕조실록), English introduction. 1,893 volumes, 49,646,667 characters, covering 472 years (1392–1863). UNESCO Memory of the World, registered 1997. https://sillok.history.go.kr/intro/english.do.

[15] Y. Oh and H. Son (2024). Lexical borrowing in Korean: a diachronic approach based on a corpus analysis. *Corpus Linguistics and Linguistic Theory*, 20(2), 407–431. https://doi.org/10.1515/cllt-2022-0102.

[16] Huffman, D.A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9), 1098–1101.

Appendix

The complete onset, vowel, and coda tables (Tables 1–3 of Section III), derived from the training split of the news corpus (Section V). Frequencies are percentages within each positional class; codes are the prefix-free Huffman codewords. Symbols absent from the training data are floored to 0.01% so that all 11,172 syllables remain encodable.

TABLE V
Table 1: onset (*choseong*) codes.

Onset	Freq (%)	Code
ㅇ	21.6	01
ㄱ	12.57	101
ㅅ	9.97	001
ㅈ	9.74	000
ㄷ	8.57	1110
ㅎ	8.24	1101
ㄹ	6.51	1001
ㄴ	4.77	11111
ㅂ	4.73	11110
ㅁ	4.04	11000
ㅊ	2.88	10001
ㅌ	2.32	10000
ㅍ	1.97	110010
ㅋ	1.11	1100111
ㄸ	0.38	11001100
ㄲ	0.34	110011011
ㅆ	0.12	1100110100
ㅃ	0.07	11001101010
ㅉ	0.07	11001101011

TABLE VI
Table 2: vowel (*jungseong*) codes.

Vowel	Freq (%)	Code
ㅏ	19.96	00
ㅣ	15.01	110
ㅡ	12.0	100
ㅗ	9.98	010
ㅓ	9.27	1110
ㅜ	7.29	1011
ㅐ	5.9	0111
ㅕ	4.95	0110
ㅔ	4.8	11111
ㅘ	2.29	111100
ㅚ	1.37	101001
ㅝ	1.28	101000
ㅟ	1.24	1111011
ㅛ	1.21	1111010
ㅠ	1.18	1010111
ㅑ	0.76	1010101
ㅖ	0.69	1010100
ㅟ	0.59	10101101
ㅙ	0.16	101011001
ㅞ	0.05	1010110001
ㅒ	0.01	1010110000

TABLE VII
TABLE 3: CODA (*JONGSEONG*) CODES; (NONE) IS THE NULL CODA.

Coda	Freq (%)	Code
(none)	51.88	1
ㄴ	15.68	011
ㅇ	9.77	001
ㄹ	8.72	0101
ㄱ	5.2	0100
ㅁ	2.79	00011
ㅂ	2.13	00010
ㅆ	1.99	00001
ㅅ	0.7	0000011
ㅍ	0.17	00000001
ㄶ	0.15	00000000
ㄷ	0.13	000001011
ㅈ	0.13	000001010
ㄺ	0.11	000000111
ㅊ	0.11	000001000
ㅄ	0.1	000000101
ㅌ	0.1	000000110
ㅎ	0.07	0000010011
ㄲ	0.03	00000100101
9 rarer codas	≤0.01	9–12 bits

To make the format concrete and self-contained, we trace the encoding of the string 한국 2026 ("Korea 2026"), which exercises the word-length prefix, jamo decomposition, and number encoding. The tokeniser produces one word and one number; the space between them is not stored, but reinserted by the decoder. Using the tables of Appendix A and Table I, the encoding reads as in Table VIII.

TABLE VIII
ENCODING OF 한국 2026 INTO 50 BITS, READ LEFT TO RIGHT.

Bits	Source	Meaning
00	Table 0	word of two syllables
1101 00 011	Tables 1–3	한 = ㅎ + ㅏ + ㄴ
101 1011 0100	Tables 1–3	국 = ㄱ + ㅜ + ㄱ
1100	Table 0	NUMBER marker
00000100	raw	character count = 4
0010 0000 0010 0110	raw	digits 2, 0, 2, 6

The complete bitstream is the 50-bit concatenation 00 11010001110110110100110000000100001000000010011 0. Against UTF-8's 88 bits (11 bytes) for the same string, .sillok uses 50 bits; with the 7-byte header, a standalone file is 14 bytes. A decoder beginning in the WORD-START state reads 00 as a two-syllable word, decodes two onset-vowel-coda triples into 한 and 국, then reads 1100 as the NUMBER marker followed by a 4-character count and four binary-coded-decimal digits, reinserting one space before the number.